

Saturday Game Guide

Product specification and implementation assignment

A game companion built for curiosity, better viewing, and evidence-backed football questions.

PRODUCT OWNER: ChatGPT

IMPLEMENTATION OWNER: You

PLAN: 10 weeks · approximately 6 hours per week

FIRST REVIEW: October 12, 2026

Version 1.0 • October 6, 2026 • Personal college football fan project

1. Product purpose and ownership

Build a personal game companion that makes watching college football more interesting. Before kickoff, it should explain how the teams play, identify players worth following, and suggest specific things to observe. After the game, it should help compare those expectations with what happened.

The central product question is: “What should I pay attention to in this game, and what evidence makes it interesting?” Financial gain and betting advice are outside the project purpose. Do not ingest odds or build betting recommendations, picks, bankroll tools, or wagering features.

You are the developer and football observer. You own research, design choices, implementation, tests, and deployment. ChatGPT is the product owner in our check-ins: I prioritize the backlog, clarify requirements, review demonstrations against acceptance criteria, make scope decisions, and help maintain a realistic pace. I will not write implementation code unless you explicitly change that arrangement. You may ask for conceptual explanations, documentation pointers, debugging questions, or a review of your reasoning without surrendering ownership of the work.

We will use a weekly asynchronous or conversational review. A reminder starts the review; it is not a booked calendar meeting or unattended supervision. Progress is assessed from the evidence you bring. A document, demonstration, repository link, screenshots, test output, or issue summary gives us something concrete to discuss. I cannot independently inspect a private repository without access.

Working assumptions: one developer, about six hours per week, local development first, ten one-week iterations. Begin with two teams and one historical matchup; expand to four to eight tracked teams and up to three weekly guides. Use 2025 as the initial completed-season candidate, subject to a coverage check, then add completed 2026 games. Calendar deadlines are provisional; acceptance gates control progression.

Success means using the guide during at least three games, identifying something new to watch in each, and keeping a brief postgame observation. Technical completeness alone is insufficient. The application should serve the viewing experience.

2. Scope and the first release

The minimum useful release includes a weekly game list, a matchup guide, comparable offensive and defensive tendencies, two or three players to watch per team, sourced coaching/scheme context where available, kickoff weather, freshness labels, and personal postgame notes.

Each guide follows this reading order: game and kickoff details; why the matchup is interesting; when each team has the ball; players to watch; three keys or watching questions; weather; coaching context; sources and sample sizes; personal observations. Make it readable on a phone and printable from the browser. A first-time reader should understand it without knowing EPA terminology.

Each key must contain: an observation to watch, a supporting metric or sourced qualitative claim, the comparison window, the sample size, and a limitation. Example template: “Watch whether Team A stays out of third-and-long. Its early-down success rate is X over N eligible plays; Team B allows Y. This describes prior opponents and does not establish a causal matchup advantage.” X and Y are placeholders, never publishable data.

Release acceptance: three selected games render end to end; every displayed statistic has a definition and denominator; every scheme assertion has a source or is labeled a personal observation; unavailable features display ‘unavailable’ rather than invented values; refreshing an imported game does not create duplicates; a cached guide survives provider downtime; pregame analysis excludes future information; one game is watched using the guide and reviewed afterward.

Defer national coverage, live play-by-play, automated film classification, comprehensive injury feeds, route metrics, pressure metrics, personnel frequency charts, opponent-adjusted player rankings, ML predictions, and multi-user accounts. These can enter the backlog only after their inputs are verified and the fan value is demonstrated.

3. Data sources and feasibility gates

Source checks were performed against official documentation on October 6, 2026. Access, coverage, and schemas can change. Documentation demonstrates a possible capability; a successful sample pull demonstrates that your account and chosen seasons support it.

CollegeFootballData (CFBD) is the primary API candidate for games, teams, venues, rosters, player/team statistics, plays, and head-coach records. Its published free tier currently permits 1,000 calls per month. Budget below that limit and cache by endpoint and parameters. API keys stay server-side. An accessible API is not automatically an unrestricted open-license database; record provider terms and dataset permissions before redistributing raw data. [1-3]

SportsDataverse/cfbfastR is the candidate bulk historical source. Its documentation describes prebuilt season release assets and multiple loader families, including classic FBS play-by-play from 2014 onward. cfbfastR is an R package; a Python project can evaluate the underlying release files rather than adding R merely to use a wrapper. Choose one primary play-by-play source for the first release. Verify availability and schema for the exact years you need. [4]

Open-Meteo is the weather candidate. Its free service supports noncommercial use subject to limits; the pricing page currently lists 10,000 calls per day and 300,000 per month, with no uptime guarantee. Use stadium coordinates, hourly conditions, explicit units, and a cached response. Historical weather and pregame forecasts are different datasets. [5-6]

Manual curation is required for coordinator tenures and qualitative scheme context unless a separate verified source provides them. CFBD's documented coaching endpoints describe head coaches, not a complete OC/DC employment history. Start with current staff biographies and dated announcements from official athletic departments. Record role, effective dates, interim status, source, and who calls plays when known. An OC title alone does not establish play-calling responsibility. [3]

The documented ordinary CFBD play response includes situation, result, play text, and PPA. It does not establish access to per-play personnel, coverage, motion, routes, pressures, or play action. Newer passing/rushing families may add charting detail, but verify fields, years, completeness, and account access before assigning work. Earlier examples of 11/12 personnel percentages, yards per route run, pressure rates, and coverage frequencies were illustrative, not a promise of open data. [2,4]

Create a data feasibility matrix with columns: feature; source/endpoint; field names; seasons; sample count; missing rate; entitlement; terms; cost; refresh; status; fallback. Status is verified, unverified, unavailable, or deferred. No feature becomes a committed sprint story until its inputs pass this gate.

Initial calls should establish game IDs, team IDs, completed-game plays, player statistics, venue coordinates, and sample weather. If player identity or advanced metrics are incomplete, ship production totals and clear missingness first. If scheme notes are unavailable, omit the scheme card. If current-season data is delayed, show the last successful refresh and keep the historical guide usable.

4. Suggested architecture and working environment

Recommended baseline: Python ingestion and analytics; PostgreSQL storage; FastAPI for a small read API; React with TypeScript for the interface. Next.js is an alternative if you prefer its conventions; do not operate two overlapping API backends without a reason. Docker Compose can run the database and local services. Pin the versions you select in lockfiles.

Use GitHub for source control and GitHub Issues/Projects for the initial board. Jira is an acceptable replacement if practicing it is a learning goal; maintain one source of truth for tickets. Use pytest for Python checks, a frontend test runner appropriate to your chosen build tool, and GitHub Actions for CI. These are proposed tools, not accounts or repositories already created for you.

Repository areas: backend/ for API, ingestion, analytics, and migrations; frontend/ for the viewer; tests/ for meaningful fixtures and integration checks; docs/ for product decisions, metric definitions, sources, schema, and runbook; curated/ for sourced coaching and scheme records; scripts/ for developer commands; .github/workflows/ for CI. Keep provider snapshots out of git unless their size and permissions justify small fixtures. Commit configuration examples without credentials.

Data flow: provider response → immutable raw snapshot → validation and identity mapping → normalized tables → versioned metric aggregates → guide evidence → read API → viewer. Personal notes live separately from provider facts. Treat this as a single application, not a set of microservices.

Retain raw payloads with source, fetched-at timestamp, request parameters, schema version, and checksum. Store UTC timestamps internally; display kickoff in the viewer's chosen timezone. Use stable internal IDs plus source mappings. Do not join players or coaches by name alone. A player's team belongs to a season/date interval, allowing transfers without rewriting old records.

First import is manual. Add scheduled refresh only after retries and reruns work. A later cadence can refresh completed-game data Monday, selected upcoming schedules midweek, and weather close to kickoff. Your pipeline schedule is separate from our product-review reminder. Rate-limit, back off on 429 responses, cap retries, and log partial failures. Page views read cached/local data rather than issuing provider requests.

Proposed performance targets: a cached guide loads in under two seconds on the documented local test setup; one selected week refreshes within ten minutes on that setup; a failed refresh retains the last valid guide. Record fixture size and hardware with measurements. These are project targets, not provider guarantees.

5. Logical database model

Design migrations yourself from these requirements. This is a logical schema, not SQL to copy. Add indexes based on actual access patterns and enforce the listed relationships.

teams: internal team_id, canonical name, aliases. team_seasons: team_id + season, conference, classification. venues: venue_id, coordinates, timezone, roof/indoor status, source. games: game_id, season, provider week and season type, home/away IDs, kickoff UTC, venue, status, score. A game can involve an opponent outside the tracked set.

source_entities: source + entity_type + source_id → internal ID, with uniqueness on the source identity. ingestion_runs: run_id, source, filters, start/end, result, counts and errors. raw_snapshots: snapshot_id, run_id, storage path, hash, fetched_at. Imported facts retain provenance linking them to an ingestion run or snapshot.

players: player_id, display name, source identity. player_team_memberships: player_id, team_id, season, position and effective dates if available. player_game_stats: game_id + player_id + category + stat_name, value, unit, source; normalize provider categories explicitly. Missing statistics are null/unavailable rather than zero. Track completeness separately from roster membership.

plays: play_id, game_id, drive_id if available, sequence, offense/defense IDs, period, clock, down, distance, yards to goal, type, yards gained, scores, provider metric and provenance. Uniqueness includes source identity or an explicitly documented deterministic key. Keep raw text; do not parse it into unverified scheme truth. Optional drives table can follow once drive-level features are needed.

coaches: coach_id and identity. coach_tenures: tenure_id, coach_id, team_id, role (HC/OC/DC/co-role), start/end dates or season bounds, precision, interim flag, play-calling status, source URL, verified_at. In-season changes require multiple intervals. coach_scheme_notes: note_id, tenure_id, description, evidence type, source, effective interval, confidence and reviewer.

weather_snapshots: weather_id, game_id, kind (forecast/historical), fetched_at, valid time, coordinates, units, temperature, wind, gusts, precipitation, provider/model when available. guide_snapshots: guide_id, game_id, analysis_as_of, metric_version, evidence payload, freshness and generation time. guide_keys: guide_id, key_id, observation, evidence references, sample sizes, limitation and rule version.

team_metric_aggregates: team_id, offense/defense, season, filter/window, analysis_as_of, metric_version, eligible counts, values and source. player_watch_candidates: player_id, game_id or window, role, component metrics, eligibility, rationale and source. observations: note_id, game_id, key_id optional, personal note, timestamp, optional broadcast clock and follow-up question.

Integrity requirements: foreign keys for normalized entities; unique provider mappings; no negative denominators; safe null handling; rerunnable upserts; corrections update affected aggregates and guides. A pregame snapshot is preserved even after the underlying completed-game facts are updated.

6. Metric definitions and evidence standards

Write a metric dictionary before creating visualizations. Each definition includes eligible plays, exclusions, numerator, denominator, units, source, as-of cutoff, missingness handling, and version. Matchup comparisons must use the same definitions and windows on both sides.

Initial valid scrimmage set: verified rush/pass plays with usable situation and result; exclude special teams, no-plays, kneels and spikes when reliably identified. Document penalty treatment and unidentified-play handling. Preserve sacks: an eligible sack counts as a dropback rather than a designed run. Scrambles and unknown play classifications need explicit mapping; do not assume official rushing totals measure designed rushing tendency.

Success rate: on eligible first downs, gain at least 50% of yards to go; on second downs, at least 70%; on third/fourth downs, convert the distance. A verified offensive touchdown is successful. Invalid or missing distance is excluded and counted as missing. These are chosen project conventions, not universal football definitions.

Early-down dropback rate: eligible first/second-down dropbacks divided by eligible first/second-down rushes plus dropbacks. Include sacks; include identifiable scrambles as dropbacks if the source supports that classification. If it cannot, label the available measure ‘recorded pass-play rate’ and disclose the limitation. No inference about personnel or coverage follows from this rate alone.

Explosive rate: initially a run of at least 10 yards or a pass of at least 20 yards, divided by eligible plays of that type. Stuff rate: eligible non-kneel run plays gaining zero or fewer yards divided by eligible run plays. Name these thresholds in the UI. Defense uses the same definitions applied to opponent plays.

EPA/PPA: use one verified provider metric with its actual name and methodology link. CFBDB PPA and another source’s EPA must not be silently treated as interchangeable. Report mean only over nonmissing values and show both eligible and metric-covered play counts. Do not train an expected-points model in the first release.

Windows: season-to-date and last three completed games, ending strictly before the guide’s analysis_as_of. Allow historical comparison to prior seasons but show them separately. Suppress ranked or strong comparative statements for fewer than 50 eligible plays in a split; show a low-sample label instead. This is a product guardrail, not a statistical guarantee. Rankings are within the tracked sample unless a complete comparison population is ingested.

Players to watch: start with separate position-aware cards, not one opaque cross-position Impact Score. QB: attempts/dropbacks and production; RB: carries and rushing production; receiver: receptions and receiving production; defender: available tackles, tackles for loss, sacks and passes defended, using provider definitions. Receptions are not targets; attempts are not snaps; sacks are not pressures. Explain the observable reason a player matters and acknowledge shared team context.

Version two can add verified usage, EPA and explosive contributions, with documented weights, missing-data behavior and sensitivity checks. Avoid multiplying loosely defined rates together: it can double-count usage, behave badly around negative EPA, and obscure interpretation. Do not assign the same play’s entire EPA independently to every participant and then sum it as team contribution.

Coordinator comparisons are descriptions of teams during a tenure. Label seasons, opponents, rule/model version and sample sizes; do not attribute a causal coaching effect. Weather comparisons need

contemporaneous forecasts for historical pregame replay, roof status, time alignment and counts of games, not just thousands of correlated plays. Wind does not by itself demonstrate a weather-caused decline in passing.

7. Read API and interface requirements

All routes below are your application contracts, not CFBD endpoints. Implement the smallest subset needed by the current sprint. Return stable IDs, explicit units, nullable unavailable fields, source metadata and freshness. Use a consistent error format.

GET /health: service state, database connectivity and last successful ingestion. GET /games?season=&week=&season_type=: game list with status, kickoff, teams and guide availability. GET /games/{id}/guide?as_of=: composed guide with analysis cutoff, metrics, watch players, keys, coaching notes, weather and source references. Reject unsupported cutoff values or identify the actual snapshot returned.

GET /teams/{id}/profile?season=&as_of=: offense/defense summaries with window and eligible counts. GET /players/{id}/profile?season=&as_of=: position-aware production and membership context. GET /coaches/{id}/history: sourced tenure list, completeness status and optional descriptive aggregates. GET /metrics/definitions: versioned dictionary.

GET /games/{id}/observations and POST /games/{id}/observations: private personal notes with optional key reference. Add editing/deletion if needed for actual use. If deployed outside your machine, restrict notes and mutation routes to you before making the service reachable. Start with a curated import workflow rather than building an administrative UI.

Guide interface acceptance: kickoff timezone is visible; unknown kickoff displays TBD; postponed/cancelled games display the correct status; no-data cards remain understandable; all charts have plain-language explanations; negative efficiency metrics are valid; print output includes sources and date; mobile widths around 375 pixels remain usable. An 'as of' label explains when inputs were current.

Generation initially uses deterministic templates and auditable rules. Produce three useful keys when evidence supports them; fewer is acceptable when data is insufficient. Qualitative 'watch whether' questions may supplement numeric keys with clear evidence labels. Never fill a required count with unsupported claims. Automated narrative generation is an optional later decision, not a first-release dependency.

8. Ten-week roadmap and dependencies

Weekly capacity is six hours: two 90-minute work sessions, one two-hour build/demo session, and one hour for review and planning. Preserve time to watch football. Estimates below total about 60 hours and include research, implementation and validation; they are planning assumptions to revise together.

Week 1 — Oct 6–12: discovery and repository foundations. Choose two teams and one completed historical game; validate sources; create the board, charter, feasibility matrix and metric dictionary draft. Demo actual samples and missingness. Gate: a feasible data path and bounded first slice. First review: Monday Oct 12.

Week 2 — Oct 13–19: reliable ingestion slice. Import teams, game, venue and player statistics; retain raw evidence; create migrations and provider identity mapping. Demo rerunning the import and a failed request. Gate: consistent IDs and no duplicate normalized facts.

Week 3 — Oct 20–26: play normalization and metrics. Classify play types, validate success/explosive/dropback conventions, calculate offense/defense summaries and record exclusions. Demo a hand-calculated fixture alongside the result. Gate: traceable calculations and honest coverage.

Week 4 — Oct 27–Nov 2: first usable viewer. Build game list and a one-game guide using existing data. Add production-based players to watch and personal notes. Gate: use the guide while watching one game, or a replay if the selected live matchup is unavailable. This is the first usable product, not a polished national dashboard.

Week 5 — Nov 3–9: evidence-backed keys and refinement. Add deterministic keys, plain-language metric help, time cutoffs and mobile/print layout. Review a before/after guide and the first postgame notes. Gate: a fan can follow each key's evidence and observe it on screen.

Week 6 — Nov 10–16: coaching and scheme context. Curate four to eight staff/tenure records and a small set of sourced scheme notes for selected teams. Handle an interim or midseason transition fixture. Gate: team context is not mislabeled as per-play classification or a causal coach rating.

Week 7 — Nov 17–23: weather and freshness. Add venue-aligned forecasts, historical context where justified, indoor/TBD behavior and stale-data states. Gate: provider failures preserve a readable guide; forecast and historical values are clearly distinct.

Week 8 — Nov 24–30: repeatable weekly operation. Expand to four to eight teams, up to three weekly guides, and a completed-game refresh command. Add run summaries, retries and correction handling. Gate: refresh succeeds, reports partial failures and stays within the measured request budget.

Week 9 — Dec 1–7: hardening and optional private deployment. Finish CI, integration checks, backup/restore, documentation and access restriction for notes. Deployment is optional; local operation satisfies the product goal. Gate: a clean checkout reproduces a sample guide and a backup restores usable data.

Week 10 — Dec 8–14: fan trial and release review. Complete three total viewing trials, review what added value, resolve critical issues and write the next backlog. Use postseason games or replays if appropriate. Gate: release criteria in section 2 pass, including a guide made from actual data.

Dependencies: verified data → reliable identity/import → metric definitions → comparable metrics → guide/keys. Scheme context needs sourced tenure facts, not the entire analytics pipeline. Weather needs verified venue/time information. Scheduled refresh follows idempotent imports. Deployment follows access

protection and reproducibility. If capacity falls below four hours per week, extend the timeline and keep the first slice small.

9. Backlog and sprint process

Use states Backlog, Ready, In Progress, Review and Done. Limit work in progress to one substantive story. A story is Ready when inputs, scope, acceptance and dependencies are understood. Done requires a demonstrated result, appropriate passing checks, documentation changes and no unresolved critical issue. Unverified dependencies go into a research spike, not an optimistic feature ticket.

Epic E1 — trustworthy data. FFG-001 (3h): feasibility matrix for chosen matchup; accept successful samples, field mapping, missing rates, costs/terms and explicit fallbacks. FFG-002 (2h): repository/board/charter; accept setup instructions, secret exclusion and prioritized tickets. FFG-003 (5h): normalized game ingestion; accept identity mappings, provenance and duplicate-free rerun. FFG-004 (4h): play normalization; accept a classification map and exclusions with counts.

Epic E2 — explainable analysis. FFG-005 (5h): dictionary and aggregate metrics; accept hand-verified calculations, denominator checks and no future games. FFG-006 (3h): players to watch; accept position-specific evidence and missing-field labels. FFG-007 (4h): keys; accept each claim's rule, inputs, window, sample and limitation. FFG-008 (2h): source/freshness contract; accept stale and partial states throughout the API.

Epic E3 — viewing companion. FFG-009 (7h): one-game viewer; accept game list, both sides of matchup, watch players and a readable phone layout. FFG-010 (3h): observations and print; accept saved notes linked to a game/key and a usable printed guide. FFG-011 (3h): curated tenures/schemes; accept official sources, dates, unknowns and an interim example. FFG-012 (3h): weather; accept kickoff alignment, units, forecast labels and indoor/TBD fallback.

Epic E4 — weekly reliability. FFG-013 (5h): repeatable refresh; accept bounded retries, measured request budget, partial-failure reporting and correction rebuild. FFG-014 (4h): CI and operations; accept meaningful checks, clean setup, backup/restore and private mutation protection if deployed. FFG-015 (7h): fan trials and release; accept three viewing notes, release checklist and a prioritized next iteration. Estimates total 60h; split any story into one- or two-session tasks before starting.

Create issues with these fields: ID/title; user value; expected behavior; explicit exclusions; acceptance criteria; evidence needed; dependency IDs; estimate; implementation notes written by you; result/demo link. Make acceptance observable. "Build analytics" is not a reviewable ticket; "success rate matches the hand-calculated eligible-play fixture" is.

Use short feature branches and small pull requests linked to issues. Explain the behavior, decisions and validation in each PR. CI checks formatting/linting, focused tests and the frontend build where applicable. Review a concrete demo at the weekly check-in, then merge passing work according to your own repository rules. Keep a decision log and release notes even though you are the only developer.

10. Validation, operations and first assignment

Metric fixtures must include first-down threshold equality, second-down threshold equality, third/fourth conversions, a touchdown, an eligible sack, a no-play penalty, kneel/spike, missing distance and missing PPA/EPA. Calculate expected answers by hand and explain the denominator. Test meaningful outcomes, not copies of implementation logic.

Integration scenarios: ingest the same game twice; import a provider correction; fail one source after another succeeds; receive a 429 then recover within a retry cap; encounter unknown team/player identity; handle a transferred player and a staff transition; refresh across UTC/local date boundaries; show TBD kickoff; handle an indoor venue; retrieve a historical guide without later-game leakage; restore a database backup. Live provider calls do not belong in routine CI; use permitted small fixtures with provenance.

End-to-end release check: choose a week → open selected game → see both offensive/defensive profiles → read evidence for a key → identify a player to watch → record observation → refresh safely → print the guide. Preserve an earlier pregame snapshot after adding results. Record actual timings for the local performance targets.

Runbook: setup; credentials; database migrations; first import; selective refresh; retries/failures; quota monitoring; source schema change; correction rebuild; backup and restore; private deployment if used. Back up the database and curated notes; retain enough raw evidence to reproduce a guide. The observation log and source curation are especially valuable because they cannot be reconstructed from a stats API alone.

Assignment 1 — due at the first review, provisionally October 12: spend about six hours on a thin discovery slice. Select two teams and one completed 2025 matchup, or another completed game with verified coverage. Write a one-page charter in your own words. Create a repository and board. Obtain credentials privately where needed. Pull the smallest actual samples for game metadata, plays and player stats; verify venue and weather access. Document field names and missingness. Write the feasibility matrix, a rough hand-drawn or digital guide wireframe, and five next issues with acceptance criteria.

For the first assignment, do not build every table or ingest years of national data. Deliver discovery evidence sufficient to decide whether the first guide can work. At least one candidate advanced feature must be explicitly verified, rejected or deferred, with a reason. A documented 'unavailable' is a useful discovery result.

First-review agenda: five minutes on fan value; ten minutes on source findings and a sample walkthrough; five minutes on blockers and definitions; five minutes on what to commit next. Bring: completed issue IDs, demo/sample, checks, actual hours, one football question, blockers, and your proposed next slice. I respond with Accepted/Changes needed/Deferred for demonstrated items, concrete requirement decisions, and a bounded next sprint.

Missed-week rule: report what changed; re-estimate remaining work; reduce scope or extend the date. Do not add compensating workload automatically. New ideas go into the backlog until they displace an existing commitment. Product owner priority: usable guide → reliable facts → understandable comparisons → sourced context → breadth.

Reference links and provenance

[1] CFBD access tiers, checked October 6, 2026: <https://collegefootballdata.com/api-tiers>

[2] CFBD ordinary plays schema, checked October 6, 2026: <https://api.collegefootballdata.com/api/plays>

[3] CFBD coaches schema, checked October 6, 2026: <https://api.collegefootballdata.com/api/coaches>

[4] cfbfastR official project documentation and loader coverage, checked October 6, 2026:
<https://cfbfastr.sportsdataverse.org/index.html>

[5] Open-Meteo free/noncommercial access limits, checked October 6, 2026:
<https://open-meteo.com/en/pricing>

[6] Open-Meteo historical forecast distinction: <https://open-meteo.com/en/docs/historical-forecast-api>

The definitions, architecture, backlog estimates, sample guardrails and performance targets in this document are product decisions. They are not claims that a provider delivers every proposed field. No production application, external repository, issue board, or data ingestion has been created as part of this specification.